Program 1:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv('california_housing.csv')

data.hist(bins=30, figsize=(15, 10))
plt.suptitle('Histograms of Numerical Features')
plt.show()

plt.figure(figsize=(15, 10))

for i, column in enumerate(data.columns):
    plt.subplot(3, 3, i+1)
    plt.boxplot(data[column])
    plt.title(column)
plt.suptitle('Box Plots of Numerical Features')
plt.show()

q1 = data.quantile(0.25)
q3 = data.quantile(0.75)
iqr = q3 - q1
outliers = ((data < (q1 - 1.5 * iqr)) | (data > (q3 + 1.5 *iqr))).sum()

print(outliers)
```

Program 2:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
data = pd.read_csv('california_housing.csv')
cor_mat = data.corr(method='pearson')
print(cor_mat)
sns.heatmap(cor_mat, annot=True)
plt.show()
sns.pairplot(data)
plt.show()
```

Program 3:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
data = pd.read_csv("iris.csv")
X = data.iloc[:, 1:5]
y = data.iloc[:, 5]
X = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
pca_df = pd.DataFrame(X_pca, columns=['PC1','PC2'])
pca_df['species'] = y
colors = {
'Iris-setosa': 'red',
'Iris-versicolor': 'green',
'Iris-virginica': 'blue'
}
plt.figure(figsize=(8,6))
plt.scatter(pca_df['PC1'],pca_df['PC2'],c=pca_df['species'].map(colors))
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
plt.title("PCA of Iris Dataset")
plt.show()
```

Program 4:

```
import pandas as pd
data=pd.read_csv('sport_preferences.csv')
attributes=data.iloc[:, 1:-1].values

target=data.iloc[:,-1].values
hypothesis=None

def find_s():
    for i in range(len(target)):
        if target[i] == "Yes":
            hypothesis=attributes[i].copy()
            break
    for i in range(len(target)):
        if target[i] == "Yes":
            for j in range(len(hypothesis)):
                if hypothesis[j] !=attributes[i][j]:
                    hypothesis[j] = '?'
            print(hypothesis)
    return hypothesis
final_hypothesis = find_s()

print("Most specific Hypothesis :", final_hypothesis)
```

Program 5:

```
import numpy as np
import pandas as pd
data = np.random.rand(100)

X_train = data [: 50]
y_train = np.array([1 if x <= 0.5 else 2 for x in X_train])
X_test = data [50:]



def knn_predict (X_train, y_train, x_test, k) :
    distances = np.abs (X_train - x_test)
    nn_indices = distances.argsort () [: k]
    nn_labels = y_train[nn_indices]
    return int (np.round (np.mean (nn_labels)))

k_values = [1, 2, 3, 4, 5, 20, 30]
for k in k_values:
    y_pred = [knn_predict(X_train,y_train,x,k) for x in X_test]
    print("\n==================================")
    print("k =",k)
    df = pd.DataFrame({
        "X_value":X_test,
        "predicted_label": y_pred})
    print(df.head())
```

Program 6:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
np.random.seed(0)
X = np.linspace(0, 2*np.pi, 100)
y = np.sin(X) + 0.1*np.random.randn(100)

def weight(x, xi, tau):
    return np.exp(-(x - xi)**2 / (2 * tau**2))

def lwr(x0, X, y, tau):
    weights = np.array([weight(x0, xi, tau) for xi in X])
    return np.sum(weights * y) / np.sum(weights)

x_test = np.linspace(0, 2*np.pi, 100)
tau = 0.5
y_pred = np.array([lwr(x, X, y, tau) for x in x_test])
plt.figure(figsize=(8,5))
plt.scatter(X, y, color='red', label='Training Data')
plt.plot(x_test, y_pred, color='blue', label='LWR Curve', linewidth=2)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Locally Weighted Regression')
plt.legend()
plt.show()
```

Program 7:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

```
def linear_regression_boston():
    housing = pd.read_csv("Boston_housing_dataset.csv")
    x = housing[["RM"]]
    y = housing["MEDV"]
    x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
    model = LinearRegression()
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)

    plt.scatter(x_test, y_test, color="blue", label='Actual')
    plt.plot(x_test, y_pred, color='red', label='Predicted')
    plt.xlabel('Average number of rooms (RM)')
    plt.ylabel('Median value of homes (MEDV)')
    plt.title('Linear Regression - Boston Housing Dataset')
    plt.legend()
    plt.show()

    print('Linear Regression - Boston Housing Dataset')
    print('Mean Squared Error:', mean_squared_error(y_test, y_pred))

linear_regression_boston()
```

Program 8:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn import tree
```

```
data = load_breast_cancer()
x = data.data
y = data.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
clf = DecisionTreeClassifier()
clf.fit(x_train, y_train)
y_pred = clf.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy * 100)
new_sample = x_test[:1]
prediction = clf.predict(new_sample)
prediction_class = "Benign" if prediction == 1 else "Malignant"
print("Predicted Class for the new sample:", prediction_class)
plt.figure(figsize=(12,8))
tree.plot_tree(clf, filled=True, feature_names=data.feature_names,
class_names=data.target_names)
plt.title("Decision Tree - Breast Cancer Dataset")
plt.show()
```

Program 9:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

images = np.load("olivetti_faces.npy")
x = images.reshape((400, -1))
y = np.repeat(np.arange(40), 10)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3)
```

```
model = GaussianNB()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred) * 100)
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(x_test[i].reshape(64,64), cmap='gray')
    plt.title(f"T:{y_test[i]} P:{y_pred[i]}")
    plt.axis('off')
plt.show()
```